



UNIVERSITÀ
DI PAVIA

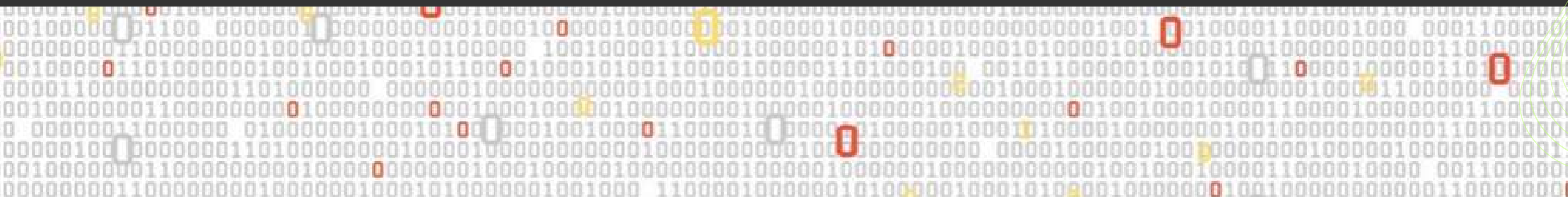
CryptoLocker Ransomware: How the Attack Works

Team 1: Cammi Raffaele, Di Frisco Ramirez Michele, Piccoli Jacopo

raffaele.cammi01@universitadipavia.it

michele.difriscoramire01@universitadipavia.it

jacopo.piccoli01@universitadipavia.it



Context and motivation

Critical Threat Landscape

Ransomware represents one of the most pervasive and disruptive cybersecurity threats in modern computing infrastructure, targeting organizations across all sectors.

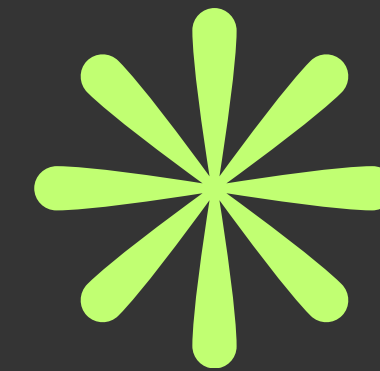
Data Availability Impact

Attacks directly compromise the availability pillar of the CIA triad, rendering critical data and systems inaccessible to legitimate users and organizations.

Understanding Attack Flow

Comprehensive knowledge of ransomware workflow enables security professionals to identify vulnerabilities and understand attack vectors systematically.

What is Ransomware?



Ransomware is a category of malicious software designed to deny access to computer systems or data until a ransom payment is made to the attacker.

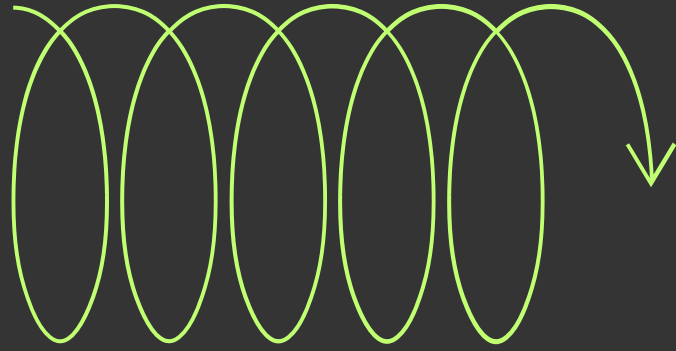
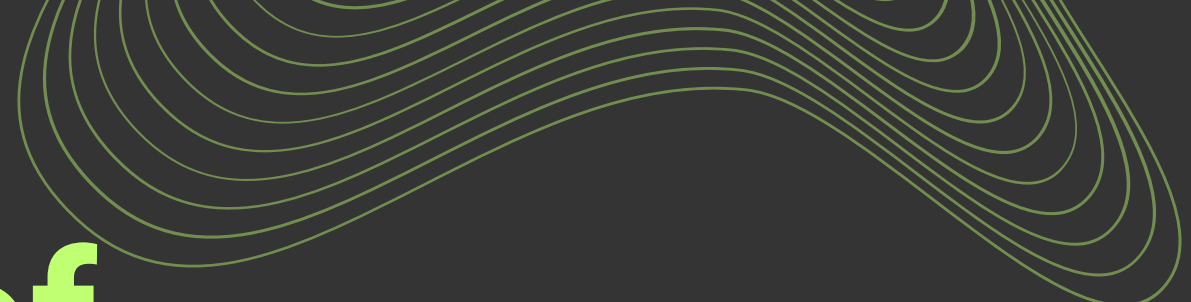
Primary Objective

- Encrypt or lock victim's files and systems
- Demand financial payment for decryption key
- Exploit urgency and data criticality

Distinction from Other Malware

Unlike information-stealing trojans or system-damaging viruses, ransomware focuses exclusively on extortion through data hostage scenarios.





Main Types of Ransomware



Locker Ransomware

Locks the victim out of their operating system entirely, preventing access to the computer interface while leaving files technically intact but completely inaccessible.

- Targets operating system functionality
- Displays full-screen ransom demands
- Restricts all user interactions

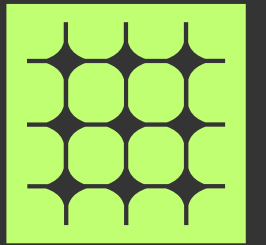


Crypto-Ransomware

Employs strong cryptographic algorithms to encrypt individual files on the system, leaving the operating system functional but rendering data unreadable without the decryption key.

- Uses asymmetric encryption methods
 - Targets specific file types and extensions
 - More prevalent in modern attacks
- 

Reference Case: CryptoLocker



Historical Significance

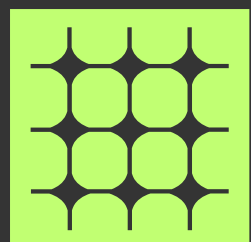
CryptoLocker, discovered in 2013, established the operational template for modern crypto-ransomware campaigns through sophisticated cryptographic implementation and distribution networks.

Target Systems

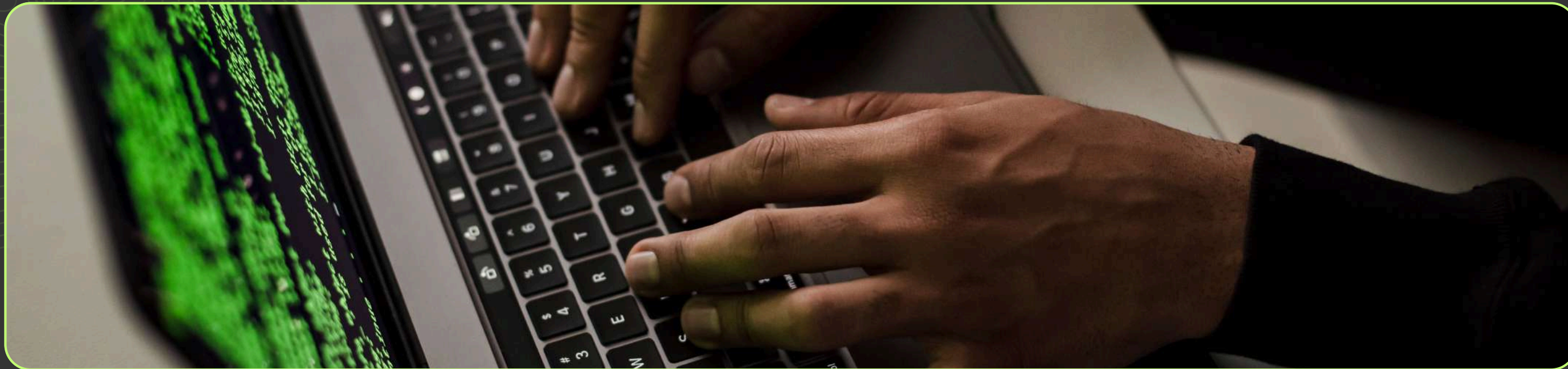
Primarily focused on Windows-based systems, exploiting vulnerabilities in desktop environments and leveraging common user behaviors for initial compromise.

Lasting Influence

CryptoLocker's technical architecture, particularly its use of asymmetric cryptography and command-and-control infrastructure, directly influenced the design patterns of modern ransomware families.



Initial Infection Phase



Common Infection Vectors

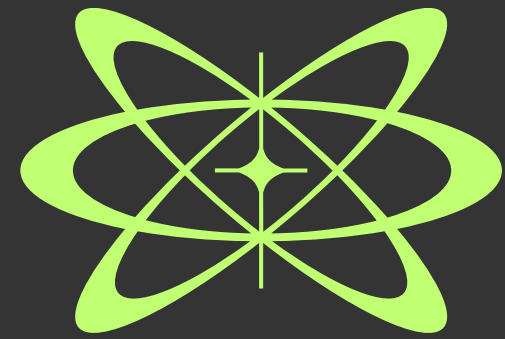
Ransomware typically infiltrates systems through malicious email attachments, compromised websites, exploit kits targeting software vulnerabilities, or infected removable media devices.

Phishing and Social Engineering

Attackers leverage psychological manipulation techniques, crafting convincing emails that appear legitimate. These messages create urgency or curiosity to prompt victims into executing malicious payloads.

Critical User Interaction

Most ransomware infections require some form of user action—opening an attachment, clicking a link, or enabling macros—highlighting the human element as a primary vulnerability in the attack chain.



Command and Control Communication



C2 Server Purpose

Command and Control servers coordinate the attack, delivering encryption keys and receiving status updates from infected systems while maintaining attacker anonymity.



Attack Coordination

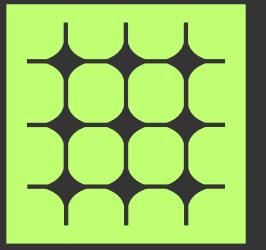
C2 servers enable dynamic control, allowing attackers to issue commands, update ransom amounts, and provide decryption services after payment verification.



Encrypted Communication

Ransomware establishes secure, encrypted channels to C2 infrastructure, evading network monitoring and preventing interception of cryptographic material by security tools.

Cryptographic Key Management



Asymmetric Cryptography

Ransomware employs public-key cryptography (typically RSA or ECC) to ensure only the attacker can decrypt files, making unauthorized recovery mathematically infeasible.

Public Key on Victim

The public key remains on the victim's system and encrypts file data. This key can only lock files but cannot reverse the encryption process.

Private Key Control

The attacker retains the private key on their infrastructure. Without this key, decryption is virtually impossible, creating absolute dependence on the attacker for data recovery.



File System Scanning and Encryption

Targeted Discovery

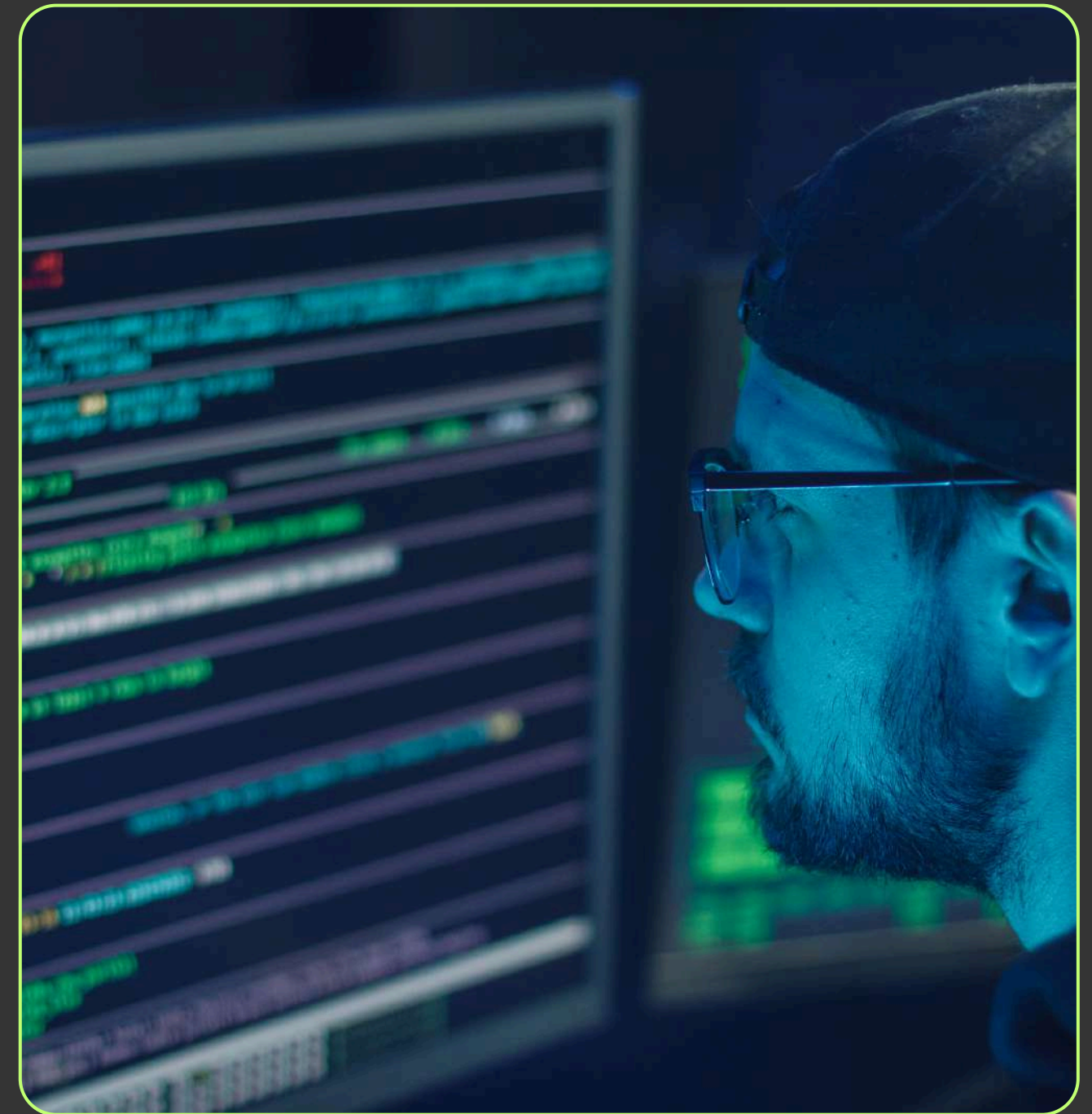
Ransomware systematically scans storage devices and network shares, identifying high-value file types (documents, databases, images, backups) based on predefined extension lists.

Encryption Process

Each identified file is encrypted using symmetric encryption (AES, for speed), with the symmetric key itself encrypted using the attacker's public key, creating a layered encryption scheme.

System Behavior

During encryption, victims may observe unusual disk activity, system slowdown, or file extension changes. The process often prioritizes critical files to maximize impact before detection.



Ransom Notification and Impact



Ransom Note Delivery

After encryption completes, the ransomware displays a prominent message—often as a changed desktop wallpaper or pop-up window—informing the victim of the attack and ransom demands.



Payment Instructions

Instructions typically specify cryptocurrency payment (Bitcoin, Monero) to maintain attacker anonymity, include specific wallet addresses, payment deadlines, and threats of permanent data loss or increased fees.

Operational Impact

Organizations face immediate operational disruption, potential data loss, regulatory compliance issues, reputational damage, and significant financial costs regardless of payment decision.



Logical Flow Summary

Initial Infection

Malicious payload delivered via phishing or compromised sites

File Discovery

Scan system for target file types across local and network storage

Encryption

Encrypt discovered files using hybrid cryptographic approach



C2 Communication

Establish encrypted channel with command and control server



Key Exchange

Receive public encryption key; private key retained by attacker



Ransom Demand

Display payment instructions and threaten permanent data loss

GameOver Zeus & CryptoLocker

The golden age of CryptoLocker was between September 2013 and May 2014. The protagonist was Zeus (discovered in 2005), specifically GameOver Zeus. This was an advanced variant with a ransomware component designed to increase the attacker's success if they couldn't steal banking credentials.

If GameOver Zeus fails to steal banking credentials, the CryptoLocker ransomware kicks in. It encrypts the victim's files using the secure RSA-2048 algorithm. Files remain inaccessible without the private key, which is only given after paying the ransom.

GameOver Zeus & CryptoLocker

Sensitive data stolen by Zeus:

- Information entered in HTTP forms during browsing.
- Data stored in Windows Protected Storage.
- Client certificates and keys used in critical public infrastructures.
- FTP and POP (email) account credentials.
- Cookies used for HTTP and Flash applications.

GameOver Zeus & CryptoLocker

Organizations can block Zeus communications using proxies and firewall rules. However, this approach could interfere with legitimate network traffic, compromising employee productivity, and moreover, might not be 100% effective. During peak activity, Zeus controlled about 1.2 million computers, and blocking IP ranges was a necessary measure in emergency situations, although risky regarding network downtime.

Technical and Business Impact

The most immediate problem caused by ransomware like CryptoLocker is the loss of file availability.

Companies suffer the greatest damage due to system downtime, resulting in the inability to provide services



CryptoLocker Detection Methods

Behavioral Analysis

This approach monitors program and file behaviour in real-time.

Heuristic Analysis

Heuristic techniques analyze code structure and program behavior to identify potential threats.

Encryption Detection

This technique directly analyzes the encryption of files or data block.



CryptoLocker Detection Methods

Machine Learning and AI

These methods involve training models on large datasets



Honeypots and Deception Technologies

These methods use traps set up to attract ransomware

Ransomware Mitigation Techniques

USER AWARENESS AND TRAINING

MULTI-FACTOR AUTHENTICATION (MFA)

BACKUP AND RECOVERY:

Maintaining multiple, isolated data copies is critical.

- **3-2-1-1 Backup:** Three total copies (primary + 2 backups), two media types, one offsite, one immutable copy.

NETWORK SEGMENTATION:

- **Limit Lateral Movement:** Isolate sensitive systems behind firewalls or VLANs. If one machine is infected, the malware is “contained” to that subnet.

Firewall and Access Controls: Enforce strict rules between segments

SANDBOXING AND MALWARE ANALYSIS:

To analyze suspicious files safely, use dynamic sandbox environments.

- **Cuckoo Sandbox:** An open-source malware analysis system. It runs files in VM(s), collects I/O, registry, and network logs. This reveals ransomware behavior (file renaming/encrypting) in a safe “laboratory.”

HONEYPOTS AND DECEPTION:

Place files with enticing names (e.g. “passwords.xlsx”) on accessible drives. Access or encryption of these files sends an immediate alert.

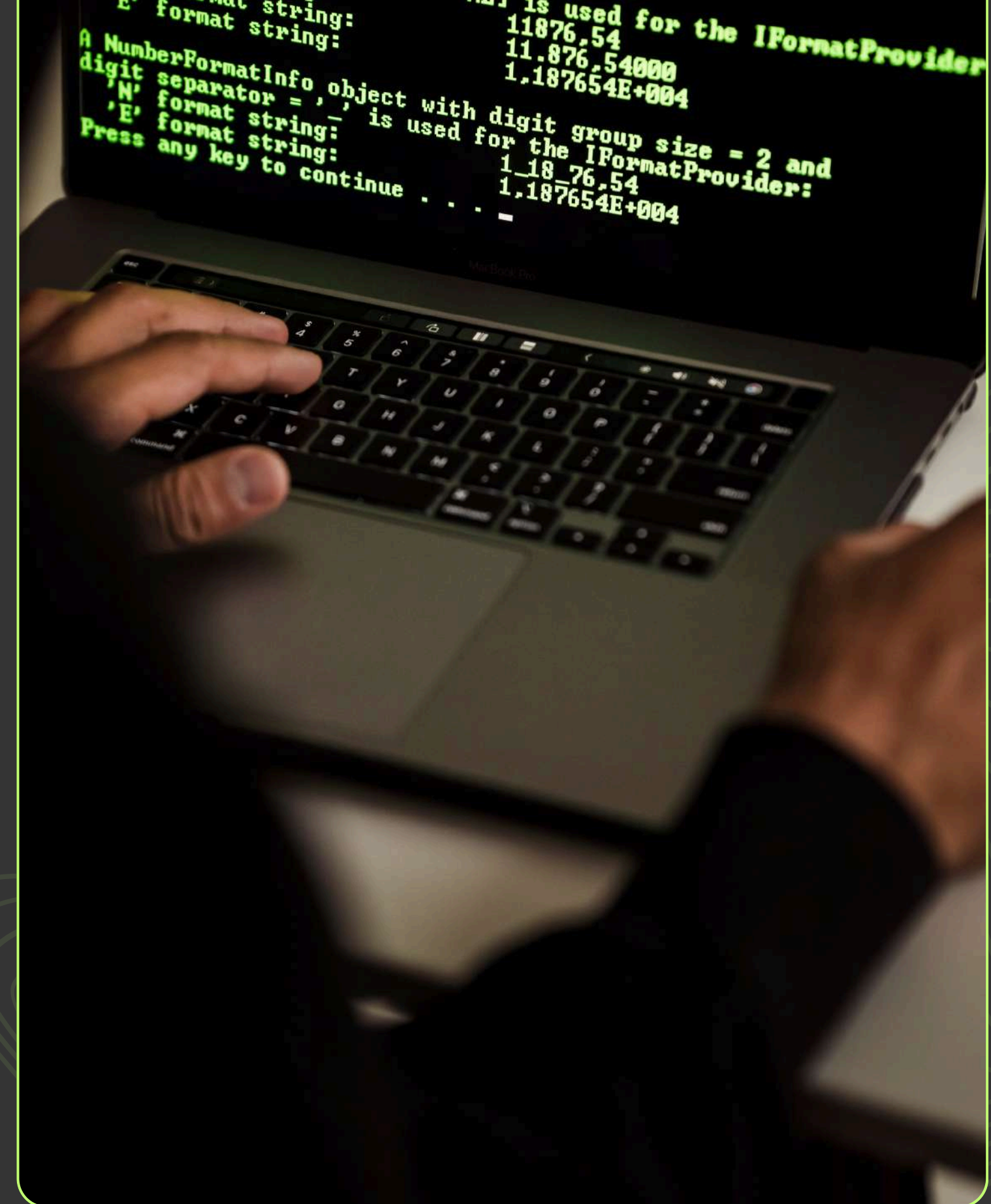
Automated Ransomware Detection

To illustrate proactive monitoring, we present two sample Python scripts.

One uses the watchdog library for real-time file-event monitoring; the other performs periodic directory scanning with `os.walk`.

Both aim to flag unusual bulk file changes (a hallmark of encryption).

As a study notes, continuously monitoring file-system activities (with Python's watchdog) can detect “suspicious patterns such as mass file modifications” in real time




```
import time, os
from watchdog.observers import Observer # type:
from watchdog.events import FileSystemEventHandl
```

Qodo: Test this class

```
class RansomwareDetector(FileSystemEventHandler)
```

Qodo: Test this method

```
def __init__(self, threshold=5):
    self.counter = {}
    self.threshold = threshold
```



This script uses watchdog to observe a directory recursively. It prints an alert if such an extension appears. It also tracks the event rate: if more than 100 file events occur within 60 seconds (indicating a burst of activity), it prints a high-rate alert. This real-time approach can provide immediate warning of fast encryption patterns

This alternate script periodically scans the directory tree. Every minute (by default), it builds a dictionary of file paths and modification times. It then compares to the previous snapshot to find any new, changed or deleted files. If the total number of changes exceeds a set threshold, it prints an alert. This polling approach is simpler and does not require external libraries.



```
import os
import time

WATCH_DIR = "C:\\Users\\migue\\OneDrive\\Desktop\\ProgectRansomware"
INTERVAL = 15
THRESHOLD = 3

Qodo: Test this function
def snapshot_directory(path):
    files_mtime = {}
    for root, dirs, files in os.walk(path):
        for name in files:
            full = os.path.join(root, name)
            try:
                files_mtime[full] = os.path.getmtime(full)
            except OSError:
                continue
    return files_mtime
```

Curiosities



actual ransom @actual_ransom · 22 apr 2019



Someone just paid 0.0584 BTC (\$309.26 USD) to a bitcoin wallet tied to #WannaCry ransomware.



blockchain.com

Address: 13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb...

The most popular and trusted block explorer and crypto transaction search engine.

THANK YOU FOR YOUR ATTENTION

Team 1: Cammi Raffaele, Di Frisco Ramirez Michele, Piccoli Jacopo



CODE 1 EXAMPLE:

```
Esempio_Watchdog.py X
C: > Users > migue > OneDrive > Desktop > Esempio_Watchdog.py > ...

1
2 import time, os
3 from watchdog.observers import Observer
4 from watchdog.events import FileSystemEventHandler
5
6 class RansomwareDetector(FileSystemEventHandler):
7     def __init__(self, threshold=5):
8         self.counter = {}
9         self.threshold = threshold
10
11     def on_modified(self, event):
12         est = os.path.splitext(event.src_path)[1]
13         # Contatore
14         self.counter[est] = self.counter.get(est, 0) + 1
15         # Threshold
16         if self.counter[est] > self.threshold:
17             print(f"WARNING: suspicious activity on the file *{est}* (Several modifications).")
18
19 if __name__ == "__main__":
20     path = "C:\\Users\\migue\\OneDrive\\Desktop\\ProgettRansomware"
21     handler = RansomwareDetector(threshold=1)
22     observer = Observer()
23     observer.schedule(handler, path, recursive=True)
24     observer.start()
25     try:
26         while True:
27             time.sleep(1)
28     except KeyboardInterrupt:
29         observer.stop()
30     observer.join()
31
```

CODE 2 EXAMPLE:

```
Esempio_Watchdog.py  Esempio_OS.py
C: > Users > migue > OneDrive > Desktop > Esempio_OS.py > ...
1  import os
2  import time
3
4  WATCH_DIR = "C:\\Users\\migue\\OneDrive\\Desktop\\ProgectRansomware"
5  INTERVAL = 5
6  THRESHOLD = 1
7
8  Qodo: Test this function
9  def snapshot_directory(path):
10     files_mtime = {}
11     for root, dirs, files in os.walk(path):
12         for name in files:
13             full = os.path.join(root, name)
14             try:
15                 files_mtime[full] = os.path.getmtime(full)
16             except OSError:
17                 continue
18     return files_mtime
19
20 previous = snapshot_directory(WATCH_DIR)
21
22 while True:
23     time.sleep(INTERVAL)
24     current = snapshot_directory(WATCH_DIR)
25     changes = []
26
27     for filepath, mtime in current.items():
28         if filepath not in previous or previous[filepath] != mtime:
29             changes.append(filepath)
30
31     for filepath in previous:
32         if filepath not in current:
33             changes.append(filepath + " (deleted)")
34
35     if len(changes) > THRESHOLD:
36         print(f"[WARNING] {len(changes)} file changes detected in the last {INTERVAL} seconds.")
37         print("Modified or deleted files:", changes)
38
39     previous = current.copy()
```